

SIMATS Engineering — Assessment Tool 2

Course: Artificial Intelligence (CSA17) | CO1: Intelligent Agents & Uninformed Search
Answer Key — Annexure A: Constraint-Based Problem Solving

Q1. Doctor-Shift Scheduling — Backtracking Search

CSP Formulation

Variables: D1, D2, D3 **Domain (each variable):** {Morning, Afternoon, Night} **Order for comparison:** Morning(1) < Afternoon(2) < Night(3)

Constraints:

- C1: D1 ≠ Night
- C2: shift(D2) < shift(D3), using the order Morning < Afternoon < Night
- C3: all-different — each shift used by exactly one doctor (one doctor per shift)
- C4: D3 ≠ Morning
- C5: every doctor is assigned exactly one shift (complete assignment / permutation)

Backtracking Search Trace (variable order D1 → D2 → D3)

Step	Assign	Constraint Check	Result
1	D1 = Morning	C1: Morning ≠ Night → OK	Proceed
2	D2 = Afternoon (Morning taken)	C3: distinct from D1 → OK	Proceed
3	D3 = Night (only shift left)	C4: Night ≠ Morning → OK. C2: Afternoon(2) < Night(3) → OK	Solution found
4	Backtrack to try D2 = Night (alt. branch)	C3: distinct from D1 → OK	Proceed
5	D3 = Afternoon (only shift left)	C2: Night(3) < Afternoon(2)? → FAIL	Backtrack
6	D1 = Afternoon (next domain value)	C1: Afternoon ≠ Night → OK	Proceed
7	D2 = Morning (Afternoon taken)	C3: distinct from D1 → OK	Proceed
8	D3 = Night (only shift left)	C4: Night ≠ Morning → OK. C2: Morning(1) < Night(3) → OK	Solution found
9	D2 = Night (alt. branch)	C3: distinct from D1 → OK	Proceed
10	D3 = Morning (only shift left)	C4: Morning = Morning → FAIL	Backtrack
11	D1 = Night	C1: Night = Night → FAIL immediately	Pruned — branch never explored

Final Valid Schedule(s)

Backtracking (depth-first, assigning D1 first) finds its **first valid solution** at step 3:

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

A second valid schedule also satisfies every constraint (found at step 8): **D1 = Afternoon, D2 = Morning, D3 = Night**. In both solutions D3 is forced into the Night shift, since C4 rules out Morning for D3 and C2 forces D3's shift to be later than D2's, which only leaves Night once D1 has taken one of the earlier shifts.

Q2. Robot Grid Navigation — Search with Manhattan Heuristic

Note: the exact 5×5 grid/obstacle layout referenced in the question was not included in the uploaded file. The representative grid below (same size, same rules) is used to demonstrate the full method — apply the identical procedure to your assignment's actual grid to get the specific answer.

Assumed Grid (rows 0–4, columns 0–4)

```
S . . . .
. # # . .
. . # . .
# . . # .
. . . . G
```

S = (0,0), G = (4,4). Obstacles (#) at (1,1), (1,2), (2,2), (3,0), (3,3). Moves: Up/Down/Left/Right, each with cost $g = 1$; obstacles and out-of-grid cells are not expanded.

Manhattan Distance Heuristic $h(n) = |r-4| + |c-4|$

For any cell (r, c), $h(n)$ estimates the remaining distance to G = (4,4) ignoring obstacles. e.g. $h(0,0) = 8$, $h(0,4) = 4$, $h(3,4) = 1$, $h(4,4) = 0$. This heuristic is admissible (never overestimates, since it ignores obstacles) and would drive node ordering in Greedy Best-First Search or A*; for plain Breadth-First Search the frontier is instead ordered purely by depth/uniform cost $g(n)$, so the table below expands nodes level-by-level while the heuristic values are reported alongside for reference.

Level-by-Level Node Expansion (BFS / priority queue ordered by $g(n)$)

Level (g)	Frontier expanded this level	$h(n)$ of each node
0	(0,0) = S	$h=8$
1	(0,1), (1,0)	$h=7$, $h=7$
2	(0,2), (2,0)	$h=6$, $h=6$
3	(0,3), (2,1)	$h=5$, $h=5$
4	(0,4), (1,3), (3,1)	$h=4$, $h=4$, $h=4$
5	(1,4), (2,3), (3,2), (4,1)	$h=3$, $h=3$, $h=3$, $h=3$
6	(2,4), (4,2), (4,0)	$h=2$, $h=2$, $h=4$
7	(3,4), (4,3)	$h=1$, $h=1$
8	(4,4) = G → GOAL REACHED	$h=0$

Cells blocked by obstacles (e.g. (1,1), (1,2), (2,2), (3,0), (3,3)) and already-visited cells are skipped during expansion and are not shown above.

Optimal Path and Cost

Tracing parent pointers back from G gives the path:

(0,0) → (0,1) → (0,2) → (0,3) → (0,4) → (1,4) → (2,4) → (3,4) → (4,4)

Total moves = 8, and since every move costs 1, **total path cost = 8** — this exactly equals the Manhattan distance $h(0,0) = 8$ between S and G, confirming the path is optimal (no obstacle forced a detour on this particular route).

Q3. Autonomous Rescue Robot — Online Search under Partial Observability

a) How the Constraints Affect Decision-Making

- **Only 4 movement directions:** restricts the branching factor of the search tree to at most 4 at every room, keeping the state space manageable but requiring careful path planning around obstacles.
- **Blocked/obstacle cells:** reduce the set of valid actions from each state and can only be discovered once sensed — the robot cannot assume a cell is passable until it has perceived it.
- **Limited sensing (adjacent cells only):** makes the environment partially observable, forcing the robot to plan and act with incomplete knowledge and revise its plan as new rooms are perceived (it cannot compute one global optimal plan up front the way an agent with full-map knowledge could).
- **Uniform move cost (1) plus risky-zone penalty (+2):** turns this into a weighted-cost search rather than a simple shortest-path count — the robot must trade off a possibly-shorter route through a risky zone against a longer but cheaper/safer route.
- **Dynamic knowledge updates (online search):** the robot must interleave acting and sensing — take a step, observe the newly-visible cells, update its internal map, and re-plan if the assumed-clear path turns out to be blocked.

b) Solution Approach — Online Uniform-Cost Search

Because the map is not fully known in advance, a pure offline UCS (which needs the complete graph upfront) cannot be run directly. Instead, an **Online Search Agent** is used, which repeatedly plans a UCS-optimal route over the *currently known* part of the grid, executes the first step, senses the newly adjacent cells, updates its internal map, and re-plans if necessary:

```
function ONLINE-RESCUE-AGENT(percept) returns action
  persistent: map (known grid so far), position, plan

  update map with percept (status of adjacent cells: free / obstacle / risky)
  if plan is empty or next step in plan is now known-blocked:
    plan <- UNIFORM-COST-SEARCH(map, position, goal)
    # edge cost = 1 for a normal move, 3 (1 + 2 penalty) for a risky-zone move
    # unexplored cells are treated optimistically (cost = 1) until sensed
  action <- FIRST(plan); plan <- REST(plan)
  return action      # e.g. move Up / Down / Left / Right
```

Each time UCS expands a node it pops the lowest cumulative-cost frontier entry from the priority queue, exactly as in Q5 of Assessment 1 — the only difference here is that the graph is built incrementally from sensor data rather than being given in full at the start.

c) Applying Core AI Concepts

- **Intelligent agent:** the robot perceives its surroundings (adjacent-cell sensing), maintains an internal model of the grid, and chooses movement actions aimed at reaching G at minimum cost — the classic percept → internal state → action loop.
- **Rationality:** at each step the robot chooses the action that, given what it currently knows, is expected to make the most progress toward minimising total cost while avoiding known hazards — it acts rationally relative to its available percepts, even though it cannot guarantee global optimality in advance because the map is incomplete.
- **Environment type:** partially observable (only adjacent cells sensed), dynamic-feeling due to incomplete knowledge even though the physical grid itself is static, sequential (each move affects future options), discrete (grid cells and discrete actions), and single-agent.

d) Justification for the Chosen Approach

An Online Search Agent built on Uniform-Cost Search is the most suitable choice because: (1) UCS naturally handles the non-uniform edge costs created by risky zones (cost 3 vs. 1), which a plain BFS could not optimise for; (2) the online variant is required — not optional — because the robot cannot see the whole building in advance (partial observability), so any offline search would need information the robot simply does not have; and (3) interleaving sensing, mapping, and re-planning lets the robot safely adapt when a presumed-clear path turns out to be blocked, which is essential for safe navigation in a disaster environment where the initial assumptions about the map may be wrong.